



MP3 Encoder

Programmer's Guide

For HiFi DSPs



Cadence Design Systems, Inc.
2655 Seely Ave.
San Jose, CA 95134
www.cadence.com

© 2024 Cadence Design Systems, Inc. All rights reserved.
Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. (Cadence) contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence's trademarks, contact the corporate legal department at the address shown above or call 1-800-862-4522.

All other trademarks are the property of their respective holders.

Restricted Print Permission: This publication is protected by copyright and any unauthorized use of this publication may violate copyright, trademark, and other laws. Except as specified in this permission statement, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, without prior written permission from Cadence. This statement grants you permission to print one (1) hard copy of this publication subject to the following conditions:

- The publication may be used solely for personal, informational, and noncommercial purposes;
- The publication may not be modified in any way;
- Any copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement;
- The information contained in this document cannot be used in the development of like products or software, whether for internal or external use, and shall not be used for the benefit of any other party, whether or not for consideration; and
- Cadence reserves the right to revoke this authorization at any time, and any such use shall be discontinued immediately upon written notice from Cadence.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. The information contained herein is the proprietary and confidential information of Cadence or its licensors, and is supplied subject to, and may be used only by Cadence's customer in accordance with, a written agreement between Cadence and its customer. Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third-party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

For further assistance, contact Cadence Online Support at <https://support.cadence.com/>.
Copyright © 2024, Cadence Design Systems, Inc. All rights reserved.

Version: 1.6

Last Updated: September 2024

Cadence Design Systems, Inc.
2655 Seely Ave.
San Jose, CA 95134
www.cadence.com

Contents

1.	Introduction to the HiFi MP3 Encoder	1
1.1	MP3 Description	1
1.2	Document Overview	1
1.3	HiFi MP3 Encoder Specifications	2
1.4	HiFi MP3 Encoder Performance	3
1.4.1	Memory	3
1.4.2	Timings	3
2.	Generic HiFi Audio Codec API	4
2.1	Memory Management	5
2.1.1	API Object	5
2.1.2	API Memory Table	5
2.1.3	Memory Types	5
2.1.3.1	Persistent Memory	5
2.1.3.2	Scratch Memory	5
2.1.3.3	Input Buffer	5
2.1.3.4	Output Buffer	6
2.2	C Language API	6
2.3	About Generic API Errors	7
2.4	Commands	9
2.4.1	Start-up API Stage	10
2.4.2	Set Codec-Specific Parameters Stage	10
2.4.3	Memory Allocation Stage	11
2.4.4	Initialize Codec Stage	12
2.4.5	Get Codec-Specific Parameters Stage	13
2.4.6	Codec Process Stage	13
2.5	Files Describing the API	14
2.6	HiFi API Command Reference	14
2.6.1	Common API Errors	15
2.6.2	XA_API_CMD_GET_LIB_ID_STRINGS	16
2.6.3	XA_API_CMD_GET_API_SIZE	19
2.6.4	XA_API_CMD_INIT	20
2.6.5	XA_API_CMD_GET_MEMTABS_SIZE	24
2.6.6	XA_API_CMD_SET_MEMTABS_PTR	25
2.6.7	XA_API_CMD_GET_N_MEMTABS	26
2.6.8	XA_API_CMD_GET_MEM_INFO_SIZE	27
2.6.9	XA_API_CMD_GET_MEM_INFO_ALIGNMENT	28
2.6.10	XA_API_CMD_GET_MEM_INFO_TYPE	29
2.6.11	XA_API_CMD_GET_MEM_INFO_PRIORITY	30
2.6.12	XA_API_CMD_SET_MEM_PTR	32
2.6.13	XA_API_CMD_INPUT_OVER	33

2.6.14	XA_API_CMD_SET_INPUT_BYTES	34
2.6.15	XA_API_CMD_GET_CURIDX_INPUT_BUF	35
2.6.16	XA_API_CMD_EXECUTE.....	36
2.6.17	XA_API_CMD_GET_OUTPUT_BYTES.....	39
3.	HiFi DSP MP3 Encoder.....	40
3.1	Files Specific to the MP3 Encoder.....	41
3.2	Configuration Parameters.....	41
3.3	MP3 Encoder-Specific Commands.....	42
3.3.1	Initialization and Execution Errors	42
3.3.2	XA_API_CMD_SET_CONFIG_PARAM	43
3.3.3	XA_API_CMD_GET_CONFIG_PARAM	47
4.	Introduction to the Example Test Bench	48
4.1	Making the Executable Files.....	48
4.2	Running Testbenches.....	49
5.	References	50

Figures

Figure 1 HiFi Audio Codec Interfaces	4
Figure 2 API Command Sequence Overview.....	9
Figure 3 Flow Chart for MP3 Encoder Integration.....	40

Tables

Table 1-1 MP3 Encoder Library Memory	3
Table 1-2 MP3 Encoder Library Runtime Memory	3
Table 1-3 MP3 Encoder Timings	3
Table 2-1 Codec API	6
Table 2-2 Error Codes Format.....	7
Table 2-3 Commands for Initialization	10
Table 2-4 Commands for Setting Parameters.....	10
Table 2-5 Commands for Initial Table Allocation.....	11
Table 2-6 Commands for Memory Allocation	11
Table 2-7 Commands for Initialization	12
Table 2-8 Commands for Getting Parameters	13
Table 2-9 Codec Processing Commands.....	13
Table 2-10 XA_CMD_TYPE_LIB_NAME Subcommand.....	16
Table 2-11 XA_CMD_TYPE_LIB_VERSION Subcommand	17
Table 2-12 XA_CMD_TYPE_API_VERSION Subcommand	18
Table 2-13 XA_API_CMD_GET_API_SIZE Command	19
Table 2-14 XA_CMD_TYPE_INIT_API_PRE_CONFIG_PARAMS Subcommand	20
Table 2-15 XA_CMD_TYPE_INIT_API_POST_CONFIG_PARAMS Subcommand.....	21
Table 2-16 XA_CMD_TYPE_INIT_PROCESS Subcommand	22
Table 2-17 XA_CMD_TYPE_INIT_DONE_QUERY Subcommand	23
Table 2-18 XA_API_CMD_GET_MEMTABS_SIZE Command	24
Table 2-19 XA_API_CMD_SET_MEMTABS_PTR Command.....	25
Table 2-20 XA_API_CMD_GET_N_MEMTABS Command.....	26
Table 2-21 XA_API_CMD_GET_MEM_INFO_SIZE Command	27
Table 2-22 XA_API_CMD_GET_MEM_INFO_ALIGNMENT Command	28
Table 2-23 XA_API_CMD_GET_MEM_INFO_TYPE Command.....	29

Table 2-24 Memory Type Indices	29
Table 2-25 XA_API_CMD_GET_MEM_INFO_PRIORITY Command	30
Table 2-26 Memory Priorities	31
Table 2-27 XA_API_CMD_SET_MEM_PTR Command	32
Table 2-28 XA_API_CMD_INPUT_OVER Command.....	33
Table 2-29 XA_API_CMD_SET_INPUT_BYTES Command	34
Table 2-30 XA_API_CMD_GET_CURIDX_INPUT_BUF Command	35
Table 2-31 XA_CMD_TYPE_DO_EXECUTE Subcommand	36
Table 2-32 XA_CMD_TYPE_DONE_QUERY Subcommand	37
Table 2-33 XA_CMD_TYPE_DO_RUNTIME_INIT Subcommand.....	38
Table 2-34 XA_API_CMD_GET_OUTPUT_BYTES Command.....	39
Table 3-1 XA_MP3ENC_CONFIG_PARAM_PCM_WDSZ subcommand	43
Table 3-2 XA_MP3ENC_CONFIG_PARAM_SAMP_FREQ subcommand	44
Table 3-3 XA_MP3ENC_CONFIG_PARAM_NUM_CHANNELS subcommand	45
Table 3-4 XA_MP3ENC_CONFIG_PARAM_BITRATE subcommand.....	46
Table 3-5 XA_MP3ENC_CONFIG_PARAM_BITRATE subcommand.....	47

Document Change History

Version	Changes
1.6	- Updated as per RJ-2024.3 tool upgrade. - Updated the performance and memory numbers in the section 1.4 HiFi MP3 Encoder Performance.
1.5	Added performance and memory data for HiFi1 in the section 1.4 HiFi MP3 Encoder Performance.
1.4	Added performance data for Fusion F1 and HiFi 3z/4/5 in the section 1.4 HiFi MP3 Encoder Performance.
1.3	- Added memory data for the HiFi Mini in the section 1.4.1 Memory. - Added timing data for the HiFi Mini in the section 1.4.2 Timings
1.2	- CPU load of the encoder has been significantly optimized. - Added new performance data for HiFi 3. - Changed generic references from “HiFi 2” to “HiFi”. - Deleted references to Diamond 330HiFi.

1. Introduction to the HiFi MP3 Encoder

The HiFi MP3 Encoder implements the audio codec standard specified in the MPEG-1 and MPEG-2 Specifications. *These standards come from the Moving Picture Experts Group (MPEG), which is a working group of ISO/IEC.*^{[1] [2]}

1.1 MP3 Description

"MPEG-1 Audio Layer III, also known as MP3, has been implemented in manifold ways. Many software packages exist to rip a track from a CD Audio and compress it in MP3. This has given rise to innovative ways of consuming music, such as the ability to create compilations to one's liking that can then downloaded to light non-mechanical MP3 players. With the arrival of MP3 the music world has been changed without recognition."

Quote From: Moving Picture Experts Group

The first MP3 player arrived in the year 1998. MPEG-2 extends MP3 to support further sampling rates and encoded bit rates. It was a success due to the better compression achieved compared to the CD format.

The target of 1.33 bits per sample results in a bitstream of 128 kbit/s for stereo 48 kHz audio. This compression is 11.7 times over the uncompressed PCM Audio.

1.2 Document Overview

This document provides all the information required to integrate the HiFi audio codecs into an application. The HiFi codec libraries implement a simple API to encapsulate the complexities of the coding operations and simplify the application and system implementation. Parts of the API described in this document are common to all the HiFi codecs. This document also details the features, provides HiFi MP3 Encoder specific information, and describes an example test bench.

1.3 HiFi MP3 Encoder Specifications

The HiFi DSP MP3 Encoder from Cadence Tensilica implements the following features in conformance to the MPEG-1 and MPEG-2 specifications:

- MPEG-2 Layer 3 Encoding
- All sampling frequencies from 16KHz to 48 kHz according to MPEG-1/2 ^{[1][2]}
- Bit rates 16-320 kbps
- Supports both mono and stereo data
- Constant bit rate (CBR)
- Joint Stereo (MS only)

The HiFi DSP MP3 Encoder does not support the following features:

- Intensity Stereo
- MPEG 2.5
- Variable bit rate (VBR)
- Average bit rate (ABR)
- Bit rate at 8 kbps

The HiFi DSP MP3 Encoder supports MP3 CBR files with a limited number of sample and data rates.

- For MPEG-1 sample rates of 32000Hz, 44100Hz, and 48000Hz, the bit rates (in kbps) are 32, 40, 48, 56, 64, 80, 96, 112, 128, 160, 192, 224, 256, 320.
- For MPEG-2 sample rates of 16000Hz, 22050Hz, and 24000Hz, the bit rates (in kbps) are 16, 24, 32, 40, 48, 56, 64, 80, 96, 112, 128, 144, 160.

The HiFi DSP MP3 Encoder implements Cadence Audio Codec API.

1.4 HiFi MP3 Encoder Performance

The HiFi DSP MP3 Encoder from Cadence Tensilica is characterized on the HiFi 5-stage DSP. The following sections provide the memory usage and performance figures for design reference.

- The API structure size returned by `XA_API_CMD_GET_API_SIZE` is approximately 600 bytes.
- The memory table structure size returned by `XA_API_CMD_GET_MEMTABS_SIZE` is approximately 140 bytes.

1.4.1 Memory

Table 1-1 MP3 Encoder Library Memory

Text (Kbytes)					Data
HiFi1	HiFi 3	HiFi 3z	HiFi 4	HiFi 5	Kbytes
48.0	478	52.2	53.5	62.4	14.2

Table 1-2 MP3 Encoder Library Runtime Memory

Runtime Memory (Kbytes)				
Persistent	Scratch	Stack	Input	Output
23.1	14.0	3.2	4.5	2.0

1.4.2 Timings

Table 1-3 MP3 Encoder Timings

Rate kHz	Channels	Bit Rate kbps	Average CPU Load (MHz)				
			HiFi1	HiFi 3	HiFi 3z	HiFi 4	HiFi 5
44.1	2	128	13.7	15.0	12.8	11.8	10.5
44.1	2	320	15.8	16.5	13.8	12.7	11.5

Note Performance specification measurements are carried out on a cycle-accurate simulator, assuming an ideal memory system with zero memory wait states. This is equivalent to running with all code and data in local memories or using an infinite-size, pre-filled cache model.

Note The MCPS numbers for HiFi 3/HiFi 3z/HiFi 4 are obtained by running the test with binary, that is, based on 24-bit optimized source code.

Note The MCPS numbers for HiFi 5 are obtained by running the test with binary, that is, recompiled from the HiFi 1's optimized source code in HiFi 5 configuration. No specific optimization is done for HiFi 5.

Note All the memory and MCPS numbers are captured with the XT-CLANG compiler and RJ-2024.3 tools.

2. Generic HiFi Audio Codec API

This section describes the API common to all the HiFi audio codec libraries. The API facilitates any codec that works in the overall method shown in **Figure 1**.

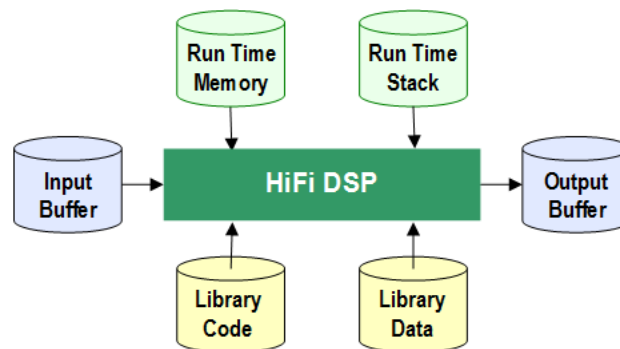


Figure 1 HiFi Audio Codec Interfaces

Section 2.1 discusses all the types of run-time memory required by the codecs. As no state information is held in static memory, a single thread can perform time division processing of multiple codecs. Additionally, multiple threads can perform concurrent codec processing. The API is implemented in a way that the application does not need to consider the codec implementation.

The codec requests the minimum sizes required for the input and output buffers through the API. Hence, before running the codec command, the codec requires that the input buffer is filled with data up to the minimum size for the input buffer. However, the codec may not consume all of the data in the input buffer. In order to properly manage the input data, the application needs to monitor the amount of data used, transfer any remaining unused data to the lower part of the input buffer, and then proceed to fill the remaining space in the buffer with new data until it reaches the minimum size again. The codec produces data in the output buffer. After the codec operation, the output data must be removed from the output buffer.

Applications that use these libraries should not make any assumptions about the size of the PCM “chunks” of data that each call to a codec produces or consumes. Normally, the chunks are the exact size of the underlying frame of the specified codec algorithm, and they will vary between codecs and also between different operating modes of the same codec. The application should provide enough data to fill the input buffer. However, some codecs provide information after the initialization stage to adjust the number of bytes of PCM data they need.

2.1 Memory Management

The HiFi audio codec API supports a flexible memory scheme and a simple interface that simplifies the integration into the final application. The API allows the codecs to request the memory required for their operations during run time.

The run-time memory requirement consists primarily of scratch and persistent memory. The codecs also require an input buffer and output buffer to transfer data into and out of the codec.

2.1.1 API Object

For each API call, the codec API stores its data in a small structure that is passed via a pointer handle to an opaque object from the application. The codec references all state information and the memory tables from this structure.

2.1.2 API Memory Table

During the memory allocation, the application is prompted to allocate memory for each of the following memory areas (described in section 2.1.3). The reference pointer to each memory area is stored in this memory table, and the reference to this memory table is stored in the API object.

2.1.3 Memory Types

2.1.3.1 Persistent Memory

This type of memory is referred to as static or context memory. It consists of the state or historical information that is preserved from one codec invocation to the next within the same thread or instance. The codecs assume that the persistent memory remains unchanged by the system, except for the codec library itself, for the entire duration of the codec operation.

2.1.3.2 Scratch Memory

This type of temporary buffer is used by the codec for processing. The contents of this memory region should be unchanged if the codec processing is active, that is if the thread running the codec is inside any API call. The system can use this region freely between successive calls to the codec.

2.1.3.3 Input Buffer

This is the buffer that the algorithm uses to accept input data. Before the call to the codec, the input buffer must be completely filled with input data.

2.1.3.4 Output Buffer

An algorithm uses this output buffer to write the output. Before the algorithm completes its tasks, the output buffer must be accessible for the codec. The application can modify the output buffer pointer between codec calls, enabling the codec to write directly to the designated output area. It is important to note that the codec will never write more data than the requested size of the output buffer.

2.2 C Language API

A single interface function is used to access the codec, with the operation specified by command codes. The API C call is defined for each codec library and is specified in the codec-specific section. Each library has a single C API call.

The C parameter definitions for every codec library are the same and are specified in the following table:

Table 2-1 Codec API

xa_<codec>	
Description	This C API is the only function that allows accessing the audio codec.
Syntax	<pre>XA_ERRORCODE xa_<codec>(xa_codec_handle_t p_xa_module_obj, WORD32 i_cmd, WORD32 i_idx, pVOID pv_value);</pre>
Parameters	p_xa_module_obj Pointer to the opaque API structure. i_cmd Command. i_idx Command subtype or index. pv_value Pointer to the variable that passes in or retrieves properties from the state structure.
Returns	Returns an error code to indicate the success or failure of the API call.

The types used for the C API calls are defined in the supplied header files as:

```
typedef signed int          WORD32;
typedef void                *pVOID;
```

Each time the C API for the codec is called, a pointer to a privately allocated data structure is passed as the first argument. This argument is treated as an opaque handle, as the application does not require the data to be looked at within the structure. The size of the structure is supplied by a specific API command so that the application can allocate the required memory. Do not use `sizeof()` for the type of the opaque handle.

Some command codes are further divided into subcommands. The command and its subcommand are passed to the codec via the second and third arguments, respectively.

When a value is passed to or returned from an API command, the expected or returned value is passed through a pointer given as the fourth argument to the C API function. When passing a pointer value to the codec, the pointer is cast to `pVOID`. In these cases, it is incorrect to pass a pointer to a pointer. For example, when the application passes the codec, a pointer is sent to an allocated memory region.

Because the operations required to decode or encode audio streams are similar, the HiFi DSP API enables the application to use the same set of procedures for each stage. By maintaining a pointer to the single API function and passing the correct API object, the same code base can be used to carry out the operations required for any of the supported codecs.

2.3 About Generic API Errors

The API functions return the error code of `XA_ERRORCODE` type, which is `signed int`. The format of the error codes is defined in Table 2-2.

Table 2-2 Error Codes Format

31	30-15	14 - 11	10 - 6	5 - 0
Fatal	Reserved	Class	Codec	Sub code

The format of error codes is explained as follows:

- The errors that are returned from the APIs are divided into fatal and nonfatal errors based on severity. Fatal errors require restarting the codec, while nonfatal errors provide information to the application.
- Errors can be further classified into the following three classes:
 - API errors occur when the API is used incorrectly.
 - Config errors occur when the codec parameters are incorrect or not supported.

- Process Errors occur during the primary encoding or decoding and indicate issues that have arisen due to invalid or bad input data.
- The subcode specifies the exact condition that triggers the error.

2.4 Commands

This section details the commands associated with the flow chart shown in Figure 2. The white blocks in the flow chart represent a common set of generic API commands, and the yellow blocks represent codec-specific commands.

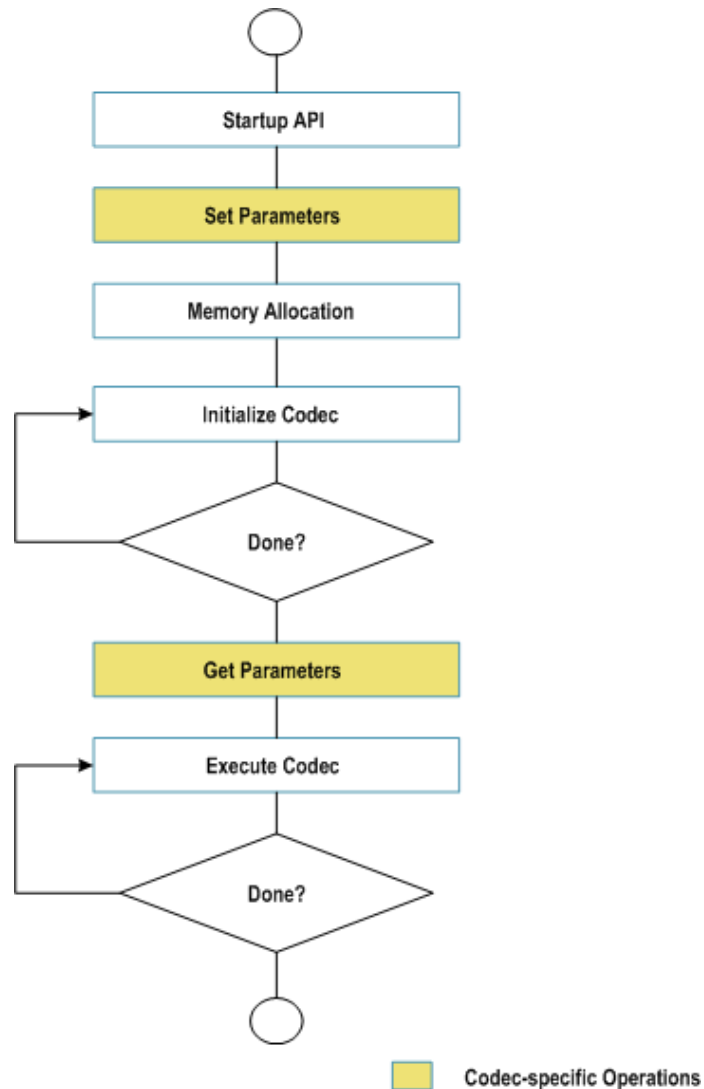


Figure 2 API Command Sequence Overview

The following sections list the commands in the order they should occur for each stage of the flow, as shown in Figure 2. For individual commands, definitions, and examples, refer to Section 2.6.

2.4.1 Start-up API Stage

The commands listed in Table 2-3 should be run once each during start-up. The commands to get the various identification strings from the codec library are optional and for information only. The command to get the API object size is mandatory as the actual object type is hidden in the library, and therefore there is no type available to use with `sizeof()`.

Table 2-3 Commands for Initialization

Command / Subcommand	Description
XA_API_CMD_GET_LIB_ID_STRINGS XA_CMD_TYPE_LIB_NAME	Gets the name of the library.
XA_API_CMD_GET_LIB_ID_STRINGS XA_CMD_TYPE_LIB_VERSION	Gets the version of the library.
XA_API_CMD_GET_LIB_ID_STRINGS XA_CMD_TYPE_API_VERSION	Gets the version of the API.
XA_API_CMD_GET_API_SIZE	Gets the size of the API structure.
XA_API_CMD_INIT XA_CMD_TYPE_INIT_API_PRE_CONFIG_PARAMS	Sets the default values of all the configuration parameters.

2.4.2 Set Codec-Specific Parameters Stage

For information on parameters that can be set, see Section 3. These parameters either control the encoding process or determine the output format of the decoder PCM data.

Table 2-4 Commands for Setting Parameters

Command / Subcommand	Description
XA_API_CMD_SET_CONFIG_PARAM XA_<codec>_CONFIG_PARAM_<param_name>	Sets the codec-specific parameter. See the codec-specific section for parameter definitions.

2.4.3 Memory Allocation Stage

The commands listed in Table 2-5 should be run once, only after all the codec-specific parameters are set. After the application allocates the memory table structure (MEMTABS) per the specified size, a pointer to the memory table structure is passed to the codec API. After the codec-specific parameters are set, the initial codec setup is completed by performing the post-configuration portion of the initialization to determine the initial operating mode of the codec and assign sizes to the blocks of memory required for its operation. The application then requests a count of the number of memory blocks.

Table 2-5 Commands for Initial Table Allocation

Command / Subcommand	Description
XA_API_CMD_GET_MEMTABS_SIZE	Gets the size of the memory structures to be allocated for the codec tables.
XA_API_CMD_SET_MEMTABS_PTR	Passes the memory structure pointer allocated for the tables.
XA_API_CMD_INIT XA_CMD_TYPE_INIT_API_POST_CONFIG_PARAMS	Calculates the required sizes for all the memory blocks based on the codec-specific parameters.
XA_API_CMD_GET_N_MEMTABS	Obtains the number of memory blocks required by the codec.

The commands listed in Table 2-6 should then be run in a loop to allocate the memory. The application first requests all the memory block attributes and then allocates them, ensuring they abide by the alignment requirements. Finally, the pointer to the allocated block of memory is passed back through the API. At this point, assigning the correct memory to the input and output buffers is unnecessary. However, the input and output buffer locations must be assigned before their first use in the Process stage. The type field refers to the memory blocks, such as input or persistent, as described in Section 2.1.

Table 2-6 Commands for Memory Allocation

Command / Subcommand	Description
XA_API_CMD_GET_MEM_INFO_SIZE	Gets the size of the memory type being referred to by the index.
XA_API_CMD_GET_MEM_INFO_ALIGNMENT	Gets the alignment information of the memory type being referred to by the index.
XA_API_CMD_GET_MEM_INFO_TYPE	Gets the type of memory referenced by the index.
XA_API_CMD_GET_MEM_INFO_PRIORITY	Gets the allocation priority of memory being referred to by the index.
XA_API_CMD_SET_MEM_PTR	Sets the pointer to the memory allocated for the referred index to the input value.

2.4.4 Initialize Codec Stage

The commands listed in Table 2-7 should be run in a loop during initialization. These commands should be called until the initialization is completed, as indicated by the `XA_CMD_TYPE_INIT_DONE_QUERY` command. In general, decoders can loop multiple times until the header information is found. However, encoders will perform exactly one call before they signal they are done.

There is a major difference between encoding Pulse Code Modulated (PCM) data and decoding stream data. During the initialization of a decoder, the initialization task reads the input stream to discover the parameters of the encoding. However, for an encoder, PCM data has no header information. Even so, the encoder application is still required to perform the initialization described in this stage. However, encoders will not consume data during initialization.

Furthermore, this implies that some encoders provide parameters that can be used to modify the input buffer data requirements after the initialization stage. These modifications will always result in a reduction in size. The application only needs to provide the reduced amount for each main codec process.

The application generally signals the codec the number of bytes available in the input buffer and signals whether it is the last iteration. It is not expected to reach the end of the data during initialization. Still, if a decoder is presented with a corrupt stream, it will allow a smooth termination. After the codec initialization is called, the application will ask for the number of bytes consumed. The application can also ask if the initialization is complete. It is advisable to always ask, even in encoders requiring only a single pass. A decoder application must keep iterating until the initialization is completed.

Table 2-7 Commands for Initialization

Command / Subcommand	Description
<code>XA_API_CMD_SET_INPUT_BYTES</code>	Sets the number of bytes available in the input buffer for initialization.
<code>XA_API_CMD_INPUT_OVER</code>	Signals the codec at the end of the bitstream.
<code>XA_API_CMD_INIT</code> <code>XA_CMD_TYPE_INIT_PROCESS</code>	Searches for the valid header, decodes header to get the parameters, and initializes state and configuration structures.
<code>XA_API_CMD_INIT</code> <code>XA_CMD_TYPE_INIT_DONE_QUERY</code>	Checks if the initialization process has been completed.
<code>XA_API_CMD_GET_CURIDX_INPUT_BUF</code>	Gets the number of input buffer bytes consumed by the last initialization.

2.4.5 Get Codec-Specific Parameters Stage

After initialization, the codec can supply the application with information. In the case of decoders, these would be the parameters it has extracted from the encoded header in the stream.

Table 2-8 Commands for Getting Parameters

Command / Subcommand	Description
XA_API_CMD_GET_CONFIG_PARAM XA_<codec>_CONFIG_PARAM_<param_name>	Gets the parameter value from the codec. See the codec-specific section for parameter definitions.

2.4.6 Codec Process Stage

The commands listed in Table 2-9 should be run continuously until the data is exhausted or the application wants to terminate the process. This is similar to the initialization stage but includes support for the management of the output buffer. After each iteration, the application requests the amount of data written to the output buffer. This amount is always limited by the size of the buffer that was requested during the memory block allocation. To alter the output buffer position, use XA_API_CMD_SET_MEM_PTR with the output buffer index.

Table 2-9 Codec Processing Commands

Command / Subcommand	Description
XA_API_CMD_INPUT_OVER	Signals the end of the bitstream to the library.
XA_API_CMD_SET_INPUT_BYTES	Sets the number of bytes available in the input buffer for the codec processing.
XA_API_CMD_EXECUTE XA_CMD_TYPE_DO_EXECUTE	Runs the codec thread.
XA_API_CMD_EXECUTE XA_CMD_TYPE_DONE_QUERY	Checks if the end of the stream has been reached.
XA_API_CMD_GET_OUTPUT_BYTES	Gets the number of bytes output by the codec in the last frame.
XA_API_CMD_GET_CURIDX_INPUT_BUF	Gets the number of input buffer bytes consumed by the last call to the codec.

2.5 Files Describing the API

Following are the common include files (`include`):

- `xa_apicmd_standards.h`
Includes command definitions for the generic API calls.
- `xa_error_standards.h`
Includes macros and definitions for all the generic errors.
- `xa_memory_standards.h`
Includes definitions for the memory block allocation.
- `xa_type_def.h`
Includes definitions of all datatypes required for the API calls.

2.6 HiFi API Command Reference

This section describes different commands along with their associated subcommands. This section does not describe the codec-specific commands. The particular codec commands are the SET and GET commands for the operational parameters.

The commands are listed below in the following sections based on their primary command type (`i_cmd`). Each section contains a table for every subcommand. If subcommands are not available, one primary command is presented.

The commands are followed by an example C call code. Along with the call, a definition of the variable types used is provided. This is to avoid any confusion over the type of the fourth argument. The examples are not complete C code extracts, as the variables are not initialized before they are used.

The errors returned by the API are listed after each of the command definitions. However, a few errors are common to all the API commands; these are listed in Section 2.6.1. The codec-specific sections will define all the errors possible from the codec-specific commands. For more information on Process errors, refer to Section 3.2.

2.6.1 Common API Errors

These errors are fatal and should not be encountered during normal application operations. They signal that a serious error has occurred in the application, calling the codec.

- **XA_API_FATAL_MEM_ALLOC**
`p_xa_module_obj` is NULL.
- **XA_API_FATAL_MEM_ALIGN**
`p_xa_module_obj` is not aligned to 4 bytes.
- **XA_API_FATAL_INVALID_CMD**
`i_cmd` is not a valid command.
- **XA_API_FATAL_INVALID_CMD_TYPE**
`i_idx` is invalid for the specified command (`i_cmd`).

2.6.2 XA_API_CMD_GET_LIB_ID_STRINGS

Table 2-10 XA_CMD_TYPE_LIB_NAME Subcommand

Subcommand	XA_CMD_TYPE_LIB_NAME
Description	This command obtains the library's name in the form of a string. The maximum length of the string that the library will provide is 30 bytes. Therefore, the application shall pass a pointer to a buffer having a minimum size of 30 bytes. This command is optional.
Actual Parameters	p_xa_module_obj NULL i_cmd XA_API_CMD_GET_LIB_ID_STRINGS i_idx XA_CMD_TYPE_LIB_NAME pv_value process name – Pointer to a character buffer in which the name of the library is returned.
Restrictions	None

Note Codec object is not required as the name is static data in the codec library.

Example

```
char process_name[30];
res = (*api_func) (NULL,
    XA_API_CMD_GET_LIB_ID_STRINGS,
    XA_CMD_TYPE_LIB_NAME,
    (pVOID) process_name);
```

Errors

- XA_API_FATAL_MEM_ALLOC
This error is suppressed as p_xa_module_obj is NULL.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

Table 2-11 XA_CMD_TYPE_LIB_VERSION Subcommand

Subcommand	XA_CMD_TYPE_LIB_VERSION
Description	This command obtains the library version in the form of a string. The maximum length of the string that the library will provide is 30 bytes. Therefore, the application shall pass a pointer to a buffer having a minimum size of 30 bytes. This command is optional.
Actual Parameters	p_xa_module_obj NULL i_cmd XA_API_CMD_GET_LIB_ID_STRINGS i_idx XA_CMD_TYPE_LIB_VERSION pv_value lib_version - Pointer to a character buffer in which the version of the library is returned.
Restrictions	None

Note Codec object is not required as the version is static data in the codec library.

Example

```
char lib_version[30];
res = (*api_func) (NULL,
                  XA_API_CMD_GET_LIB_ID_STRINGS,
                  XA_CMD_TYPE_LIB_VERSION,
                  (pVOID) lib_version);
```

Errors

- XA_API_FATAL_MEM_ALLOC
This error is suppressed as p_xa_module_obj is NULL.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

Table 2-12 XA_CMD_TYPE_API_VERSION Subcommand

Subcommand	XA_CMD_TYPE_API_VERSION
Description	This command obtains the API version in the form of a string. The maximum length of the string that the library will provide is 30 bytes. Therefore, the application shall pass a pointer to a buffer having a minimum size of 30 bytes. This command is optional.
Actual Parameters	p_xa_module_obj NULL i_cmd XA_API_CMD_GET_LIB_ID_STRINGS i_idx XA_CMD_TYPE_API_VERSION pv_value api_version - Pointer to a character buffer in which the version of the API is returned.
Restrictions	None

Note Codec object is not required as the version is static data in the codec library.

Example

```
char api_version[30];
res = (*api_func) (NULL,
    XA_API_CMD_GET_LIB_ID_STRINGS,
    XA_CMD_TYPE_API_VERSION,
    (pVOID) api_version);
```

Errors

- XA_API_FATAL_MEM_ALLOC
This error is suppressed as p_xa_module_obj is NULL.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

2.6.3 XA_API_CMD_GET_API_SIZE

Table 2-13 XA_API_CMD_GET_API_SIZE Command

Subcommand	None
Description	This command is used to obtain the size of the API structure and allocate memory for the API structure. The pointer to the API size variable is passed, and the API returns the structure size in bytes. The API structure is used for the interface and is persistent.
Actual Parameters	p_xa_module_obj NULL i_cmd XA_API_CMD_GET_API_SIZE i_idx NULL pv_value &api_size – Pointer to the API size variable.
Restrictions	The application will allocate memory with an alignment of 4 bytes.

Note A Codec object is not required as the size is fixed for the codec library.

Example

```

unsigned int api_size;
res = (*api_func) (NULL,
    XA_API_CMD_GET_API_SIZE,
    0,
    (pVOID) &api_size);

```

Errors

- XA_API_FATAL_MEM_ALLOC
This error is suppressed as p_xa_module_obj is NULL.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

2.6.4 XA_API_CMD_INIT

Table 2-14 XA_CMD_TYPE_INIT_API_PRE_CONFIG_PARAMS Subcommand

Subcommand	XA_CMD_TYPE_INIT_API_PRE_CONFIG_PARAMS
Description	This command sets the default value of the configuration parameters. The configuration parameters can then be altered by using one of the codec-specific parameter setting commands. Refer to Section 3.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API Structure. i_cmd XA_API_CMD_INIT i_idx XA_CMD_TYPE_INIT_API_PRE_CONFIG_PARAMS pv_value NULL
Restrictions	None

Example

```
res = (*api_func) (api_obj,  
                  XA_API_CMD_INIT,  
                  XA_CMD_TYPE_INIT_API_PRE_CONFIG_PARAMS,  
                  NULL);
```

Errors

- Common API Errors.

Table 2-15 XA_CMD_TYPE_INIT_API_POST_CONFIG_PARAMS Subcommand

Subcommand	XA_CMD_TYPE_INIT_API_POST_CONFIG_PARAMS
Description	This command calculates the sizes of all the memory blocks required by the application. It should occur after the codec-specific parameters are set.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API Structure. i_cmd XA_API_CMD_INIT i_idx XA_CMD_TYPE_INIT_API_POST_CONFIG_PARAMS pv_value NULL
Restrictions	None

Example

```
res = (*api_func)(api_obj,  
                XA_API_CMD_INIT,  
                XA_CMD_TYPE_INIT_API_POST_CONFIG_PARAMS,  
                NULL);
```

Errors

- Common API Errors.

Table 2-16 XA_CMD_TYPE_INIT_PROCESS Subcommand

Subcommand	XA_CMD_TYPE_INIT_PROCESS
Description	This command initializes the codec. In the case of a decoder, it searches for the valid header and performs the header decoding to get the encoded stream parameters. This command is part of the initialization loop. It must be repeatedly called until the codec signals it has finished. In the case of an encoder, the initialization of the codec is performed. No output data is created during initialization.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure i_cmd XA_API_CMD_INIT i_idx XA_CMD_TYPE_INIT_PROCESS pv_value NULL
Restrictions	None

Example

```
res = (*api_func) (api_obj,  
                  XA_API_CMD_INIT,  
                  XA_CMD_TYPE_INIT_PROCESS,  
                  NULL);
```

Errors

- Common API Errors.
- See the codec-specific section for Process errors.

Table 2-17 XA_CMD_TYPE_INIT_DONE_QUERY Subcommand

Subcommand	XA_CMD_TYPE_INIT_DONE_QUERY
Description	This command checks to see if the initialization process has been completed. If it has, the flag value is set to 1; otherwise, it is set to zero. A pointer to the flag variable is passed as an argument.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_INIT i_idx XA_CMD_TYPE_INIT_DONE_QUERY pv_value &init_done – Pointer to a flag that indicates the completion of the initialization process.
Restrictions	None

Example

```

unsigned int init_done;
res = (*api_func) (api_obj,
                  XA_API_CMD_INIT,
                  XA_CMD_TYPE_INIT_DONE_QUERY,
                  (pVOID) &init_done);

```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

2.6.5 XA_API_CMD_GET_MEMTABS_SIZE

Table 2-18 XA_API_CMD_GET_MEMTABS_SIZE Command

Subcommand	None
Description	This command obtains the table size to hold the memory blocks required for the codec operation. The API returns the total size of the required table. A pointer to the size variable is sent with this API command, and the codec writes the variable's value.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_MEMTABS_SIZE i_idx NULL pv_value &proc_mem_tabs_size – Pointer to the memory size variable.
Restrictions	The application shall allocate memory with an alignment of 4 bytes.

Example

```
unsigned int proc_mem_tabs_size;
res = (*api_func) (api_obj,
                  XA_API_CMD_GET_MEMTABS_SIZE,
                  0,
                  (pVOID) &proc_mem_tabs_size);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

2.6.6 XA_API_CMD_SET_MEMTABS_PTR

Table 2-19 XA_API_CMD_SET_MEMTABS_PTR Command

Subcommand	None
Description	This command sets the memory structure pointer in the library to the allocated value.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_SET_MEMTABS_PTR i_idx NULL pv_value alloc – Allocated pointer.
Restrictions	The application will allocate memory with an alignment of 4 bytes.

Example

```
int * alloc; //alloc is a pointer to the allocated memory
res = (*api_func) (api_obj,
                  XA_API_CMD_SET_MEMTABS_PTR,
                  0,
                  (pVOID) alloc);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.
- XA_API_FATAL_MEM_ALIGN
pv_value is not aligned to 4 bytes.

2.6.7 XA_API_CMD_GET_N_MEMTABS

Table 2-20 XA_API_CMD_GET_N_MEMTABS Command

Subcommand	None
Description	This command obtains the number of memory blocks needed by the codec. This value is used as the iteration counter for allocating memory blocks. A pointer to each memory block will be placed in the previously allocated memory tables. The pointer to the variable is passed to the API, and the codec writes the value for this variable.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_N_MEMTABS i_idx NULL pv_value &n_mems – Number of memory blocks required to be allocated.
Restrictions	None

Example

```
int n_mems;  
res = (*api_func) (api_obj,  
                  XA_API_CMD_GET_N_MEMTABS,  
                  0,  
                  (pVOID) &n_mems);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

2.6.8 XA_API_CMD_GET_MEM_INFO_SIZE

Table 2-21 XA_API_CMD_GET_MEM_INFO_SIZE Command

Subcommand	Memory index
Description	This command obtains the size of the memory type referred to by the index. The size in bytes is returned in the variable pointed to by the final argument. Note that this is the actual size needed and does not include any alignment packing space.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_MEM_INFO_SIZE i_idx Index of the memory. pv_value &size – Pointer to the memory size.
Restrictions	None

Example

```
int index;
unsigned int size;
res = (*api_func) (api_obj,
                  XA_API_CMD_GET_MEM_INFO_SIZE,
                  index,
                  (pVOID) &size);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.
- XA_API_FATAL_INVALID_CMD_TYPE
i_idx is an invalid memory block number; valid block numbers obey the relation $0 \leq i_idx < n_mems$ (See XA_API_CMD_GET_N_MEMTABS).

2.6.9 XA_API_CMD_GET_MEM_INFO_ALIGNMENT

Table 2-22 XA_API_CMD_GET_MEM_INFO_ALIGNMENT Command

Subcommand	Memory index
Description	This command gets the alignment information of the memory type being referred to by the index. The alignment required in bytes is returned to the application.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_MEM_INFO_ALIGNMENT i_idx Index of the memory. pv_value &alignment – Pointer to the alignment info variable.
Restrictions	None

Example

```
int index;
unsigned int alignment;
res = (*api_func)(api_obj,
    XA_API_CMD_GET_MEM_INFO_ALIGNMENT,
    index,
    (pVOID) &alignment);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.
- XA_API_FATAL_INVALID_CMD_TYPE
i_idx is an invalid memory block number; valid block numbers obey the relation $0 \leq i_idx < n_mems$ (See XA_API_CMD_GET_N_MEMTABS).

2.6.10 XA_API_CMD_GET_MEM_INFO_TYPE

Table 2-23 XA_API_CMD_GET_MEM_INFO_TYPE Command

Subcommand	Memory index
Description	This command gets the type of memory being referred to by the index.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_MEM_INFO_TYPE i_idx Index of the memory. pv_value &type – Pointer to the memory type variable.
Restrictions	None

Example

```
int index;
unsigned int type;
res = (*api_func)(api_obj,
    XA_API_CMD_GET_MEM_INFO_TYPE,
    index,
    (pVOID) &type);
```

Table 2-24 Memory Type Indices

Type	Description
XA_MEMTYPE_PERSIST	Persistent Memory
XA_MEMTYPE_SCRATCH	Scratch Memory
XA_MEMTYPE_INPUT	Input Buffer
XA_MEMTYPE_OUTPUT	Output Buffer

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.
- XA_API_FATAL_INVALID_CMD_TYPE
i_idx is an invalid memory block number; valid block numbers obey the relation $0 \leq i_idx < n_mems$ (See XA_API_CMD_GET_N_MEMTABS).

2.6.11 XA_API_CMD_GET_MEM_INFO_PRIORITY

Table 2-25 XA_API_CMD_GET_MEM_INFO_PRIORITY Command

Subcommand	Memory index
Description	This command gets the allocation priority of memory referred to by the index. (The meaning of the levels is defined on a codec-specific basis. Unless the codec defines it otherwise, this command returns a fixed dummy value.)
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_MEM_INFO_PRIORITY i_idx Index of the memory. pv_value &priority – Pointer to the memory priority variable.
Restrictions	None

Example

```
int index;
unsigned int priority;
res = (*api_func)(api_obj,
                  XA_API_CMD_GET_MEM_INFO_PRIORITY,
                  index,
                  (pVOID) &priority);
```

Table 2-26 Memory Priorities

Priority	Type
0	XA_MEMPRIORITY_ANYWHERE
1	XA_MEMPRIORITY_LOWEST
2	XA_MEMPRIORITY_LOW
3	XA_MEMPRIORITY_NORM
4	XA_MEMPRIORITY_ABOVE_NORM
5	XA_MEMPRIORITY_HIGH
6	XA_MEMPRIORITY_HIGHER
7	XA_MEMPRIORITY_CRITICAL

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
`pv_value` is NULL.
- XA_API_FATAL_INVALID_CMD_TYPE
`i_idx` is an invalid memory block number; valid block numbers obey the relation $0 \leq i_idx < n_mems$ (See XA_API_CMD_GET_N_MEMTABS).

2.6.12 XA_API_CMD_SET_MEM_PTR

Table 2-27 XA_API_CMD_SET_MEM_PTR Command

Subcommand	Memory index
Description	This command passes the pointer to the allocated memory to the codec. The pointer is then stored in the memory tables structure allocated earlier. For the input and output buffers, running this command during the main codec loop is legitimate.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_SET_MEM_PTR i_idx Index of the memory pv_value alloc – Pointer to the allocated memory buffer.
Restrictions	The pointer must be correctly aligned to the requirements.

Example

```
int index;
void * alloc; //alloc is a pointer to the aligned memory
res = (*api_func) (api_obj,
                  XA_API_CMD_SET_MEM_PTR,
                  index,
                  (pVOID) alloc);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.
- XA_API_FATAL_INVALID_CMD_TYPE
i_idx is an invalid memory block number; valid block numbers obey the relation $0 \leq i_idx < n_mems$ (See XA_API_CMD_GET_N_MEMTABS).
- XA_API_FATAL_MEM_ALIGN
pv_value is not of the required alignment for the requested memory block.

2.6.13 XA_API_CMD_INPUT_OVER

Table 2-28 XA_API_CMD_INPUT_OVER Command

Subcommand	None
Description	This command informs the codec that the end of the input data has been reached. This situation can arise both in the initialization and processing loop.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_INPUT_OVER i_idx NULL pv_value NULL
Restrictions	None

Example

```
res = (*api_func)(api_obj,  
    XA_API_CMD_INPUT_OVER,  
    0,  
    NULL);
```

Errors

- Common API Errors.

2.6.14 XA_API_CMD_SET_INPUT_BYTES

Table 2-29 XA_API_CMD_SET_INPUT_BYTES Command

Subcommand	None
Description	This command sets the number of bytes available in the input buffer for the codec. It is used both in the initialization and processing loop. It is the number of valid bytes from the buffer pointer. At least the minimum buffer size should be requested unless this is the end of the data.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_SET_INPUT_BYTES i_idx NULL pv_value &buff_size – Pointer to the input byte variable.
Restrictions	None

Example

```
int buff_size;  
res = (*api_func) (api_obj,  
                  XA_API_CMD_SET_INPUT_BYTES,  
                  0,  
                  (pVOID) &buff_size);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

2.6.15 XA_API_CMD_GET_CURIDX_INPUT_BUF

Table 2-30 XA_API_CMD_GET_CURIDX_INPUT_BUF Command

Subcommand	None
Description	This command gets the number of input buffer bytes consumed by the codec. It is used both in the initialization and processing loop.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_CURIDX_INPUT_BUF i_idx NULL pv_value &bytes_consumed – Pointer to the bytes consumed variable.
Restrictions	None

Example

```
int bytes_consumed;  
res = (*api_func)(api_obj,  
    XA_API_CMD_GET_CURIDX_INPUT_BUF,  
    0,  
    (pVOID) &bytes_consumed);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

2.6.16 XA_API_CMD_EXECUTE

Table 2-31 XA_CMD_TYPE_DO_EXECUTE Subcommand

Subcommand	XA_CMD_TYPE_DO_EXECUTE
Description	This command runs the codec.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_EXECUTE i_idx XA_CMD_TYPE_DO_EXECUTE pv_value NULL
Restrictions	None

Example

```
res = (*api_func) (api_obj,  
                  XA_API_CMD_EXECUTE,  
                  XA_CMD_TYPE_DO_EXECUTE,  
                  NULL);
```

Errors

- Common API Errors.
- See the codec-specific section for Process errors.

Table 2-32 XA_CMD_TYPE_DONE_QUERY Subcommand

Subcommand	XA_CMD_TYPE_DONE_QUERY
Description	This command checks to see if the processing is completed. If it has, the flag value is set to 1; otherwise, it is set to zero. The pointer to the flag is passed as an argument. Processing by the codec can continue for several invocations of the DO_EXECUTE command after the last input data has been passed to the codec. Thus, the application should not assume that the codec has finished generating all its output until the command indicates it.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_EXECUTE i_idx XA_CMD_TYPE_DONE_QUERY pv_value &flag – Pointer to the flag variable.
Restrictions	None

Example

```
int flag;  
res = (*api_func)(api_obj,  
                 XA_API_CMD_EXECUTE,  
                 XA_CMD_TYPE_DONE_QUERY,  
                 (pVOID) &flag);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

Table 2-33 XA_CMD_TYPE_DO_RUNTIME_INIT Subcommand

Subcommand	XA_CMD_TYPE_DO_RUNTIME_INIT
Description	This command resets the decoder's history buffers. It can be used to avoid distortions and clicks by facilitating playback ramping up and down during trick play. The command should be issued before the application starts feeding the decoder with new data from a random place in the input stream. Note: This command is available in API version 1.14 or later.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_EXECUTE i_idx XA_CMD_TYPE_DO_RUNTIME_INIT pv_value NULL
Restrictions	None

Example

```
res = (*api_func)(api_obj,  
                  XA_API_CMD_EXECUTE,  
                  XA_CMD_TYPE_DO_RUNTIME_INIT,  
                  NULL);
```

Errors

- Common API Errors.

2.6.17 XA_API_CMD_GET_OUTPUT_BYTES

Table 2-34 XA_API_CMD_GET_OUTPUT_BYTES Command

Subcommand	None
Description	This command obtains the number of bytes output during the last codec processing.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_OUTPUT_BYTES i_idx NULL pv_value &out_bytes – Pointer to the output bytes variable.
Restrictions	None

Example

```
int out_bytes;
res = (*api_func)(api_obj,
    XA_API_CMD_GET_OUTPUT_BYTES,
    0,
    (pVOID) &out_bytes);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

3. HiFi DSP MP3 Encoder

The HiFi DSP MP3 Encoder conforms to the generic codec API. The flow chart of the command sequence used on the example test bench is shown in Figure 3.

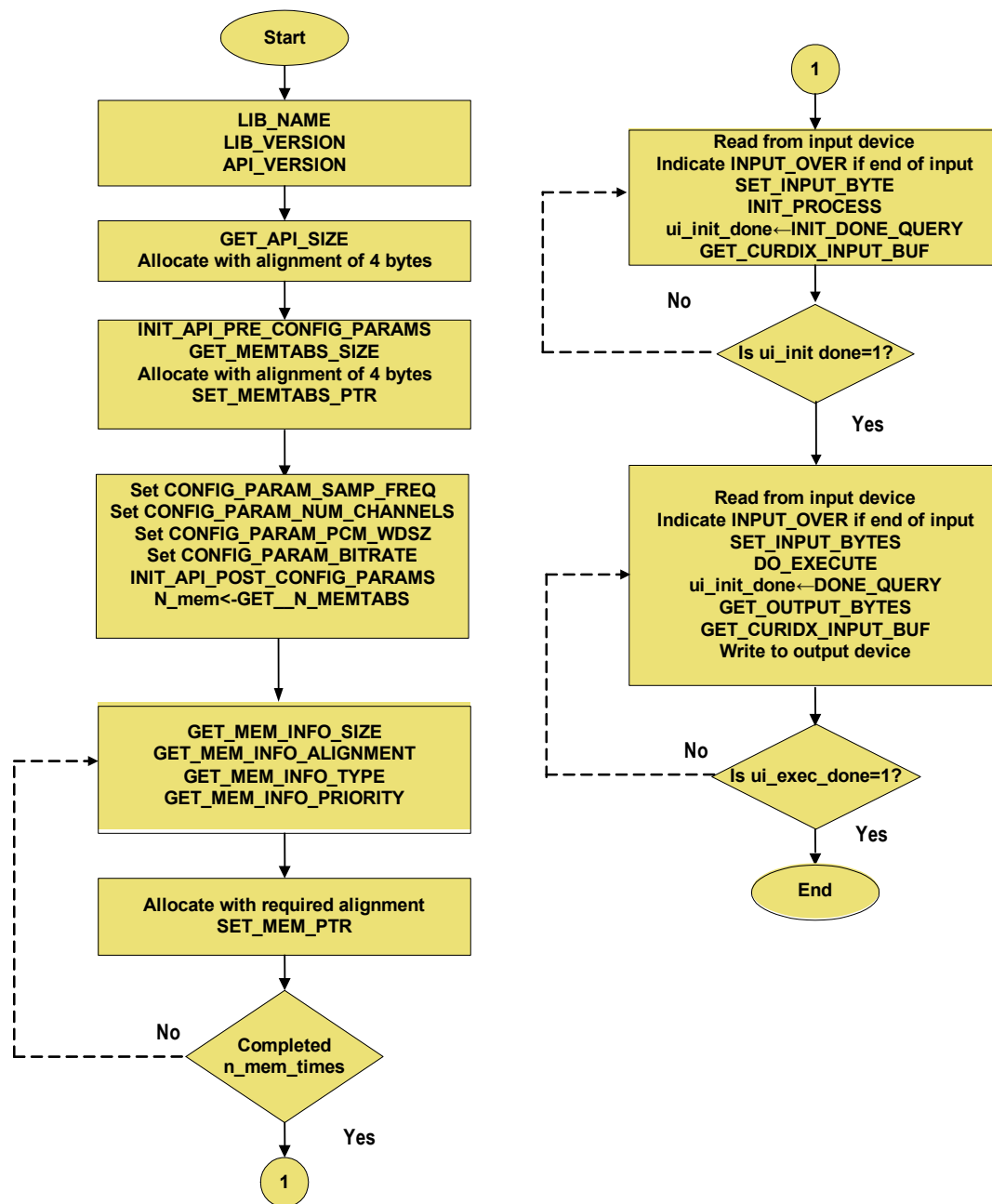


Figure 3 Flow Chart for MP3 Encoder Integration

3.1 Files Specific to the MP3 Encoder

The MP3 Encoder parameter header file (include/mp3_enc/):

- `xa_mp3_enc_api.h`

The MP3 Encoder library (lib):

- `xa_mp3_enc.a`

The MP3 Encoder API call is defined as:

```
XA_ERRORCODE xa_mp3_enc(xa_codec_handle_t p_xa_module_obj,
                        WORD32             i_cmd,
                        WORD32             i_idx,
                        pVOID              pv_value);
```

3.2 Configuration Parameters

The HiFi MP3 encode algorithm accepts the following parameters from the user.

These parameters must be supplied as no header information is encoded in the PCM data provided through the input buffer. The application must determine these parameters from the source of the data. For example, the header data can be used when using a '.wav' file.

- **samp_freq** – This is the sample frequency of the input samples. The units are Hz (samples per second). For example, a valid input is 44100.
- **num_chan** – This is the number of channels of data that are interleaved into the input buffer. There are only two valid values 1 (monaural) and 2 (stereo). In the case of stereo, the output data are interleaved in the output buffer, with the first output sample in each pair being the left channel value and the second sample in each pair being the right channel value.
- **pcm_wd_sz** – This is the PCM word size of the data samples. The valid value is 16 only.

The following parameter is required for the encoding process:

- **bit_rate** – This should be set to a valid MP3 bit rate. If not, the next higher valid rate will be selected. Note the units are in kbps; 128 is a valid example.

The encoder provides the following parameter:

- `bit_rate` – This is the rounded-up (if necessary) bit rate actually used during encoding.

3.3 *MP3 Encoder-Specific Commands*

These are the commands unique to the HiFi MP3 encoder. They are listed in sections based on their primary command type (`i_cmd`). Each section contains a table for every subcommand. In the case of no subcommands, the one primary command is presented.

3.3.1 Initialization and Execution Errors

These errors can result from the initialization or execution of API calls:

- `XA_MP3ENC_CONFIG_NONFATAL_INVALID_BITRATE`
 - During initialization, the bit rate was adjusted upwards to the next valid value

3.3.2 XA_API_CMD_SET_CONFIG_PARAM

Table 3-1 XA_MP3ENC_CONFIG_PARAM_PCM_WDSZ subcommand

Subcommand	XA_MP3ENC_CONFIG_PARAM_PCM_WDSZ
Description	This command sets the PCM Word format size of the input samples in bits.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure i_cmd XA_API_CMD_SET_CONFIG_PARAM i_idx XA_MP3ENC_CONFIG_PARAM_PCM_WDSZ pv_value &pcm_wd_sz – Pointer to sampling frequency variable
Restrictions	Valid only for 16.

Example

```

unsigned int pcm_wd_sz;
res = (*api_func)(api_obj,
                  XA_API_CMD_SET_CONFIG_PARAM,
                  XA_MP3ENC_CONFIG_PARAM_PCM_WDSZ,
                  (void *) &pcm_wd_sz);

```

Errors

- Common API Errors
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL
- XA_MP3ENC_CONFIG_FATAL_PCM_WDSZ
Value is not 16

Table 3-2 XA_MP3ENC_CONFIG_PARAM_SAMP_FREQ subcommand

Subcommand	XA_MP3ENC_CONFIG_PARAM_SAMP_FREQ
Description	Set the sampling frequency of the input PCM.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure i_cmd XA_API_CMD_SET_CONFIG_PARAM i_idx XA_MP3ENC_CONFIG_PARAM_SAMP_FREQ pv_value &samp_freq – Pointer to sampling frequency variable
Restrictions	Valid for 16000, 22050, 24000, 32000, 44100 and 48000

Example

```
int samp_freq;
res = (*api_func)(api_obj,
                  XA_API_CMD_SET_CONFIG_PARAM,
                  XA_MP3ENC_CONFIG_PARAM_SAMP_FREQ,
                  (void *) &samp_freq);
```

Errors

- Common API Errors
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL
- XA_MP3ENC_CONFIG_FATAL_SAMP_FREQ
Value not valid

Table 3-3 XA_MP3ENC_CONFIG_PARAM_NUM_CHANNELS subcommand

Subcommand	XA_MP3ENC_CONFIG_PARAM_NUM_CHANNELS
Description	Set the number of interleaved channels in the PCM input buffer. When set to 2 channels, the Joint Stereo (MS) enabling decision is made internally depending upon the input data.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure i_cmd XA_API_CMD_SET_CONFIG_PARAM i_idx XA_MP3ENC_CONFIG_PARAM_NUM_CHANNELS pv_value &num_chan – Pointer to number of channels variable
Restrictions	The value should be 1 or 2.

Example

```
int num_chan;
res = (*api_func)(api_obj,
                  XA_API_CMD_SET_CONFIG_PARAM,
                  XA_MP3ENC_CONFIG_PARAM_NUM_CHANNELS,
                  (void *) &num_chan);
```

Errors

- Common API Errors
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL
- XA_MP3ENC_CONFIG_FATAL_NUM_CHANNELS
Value not valid

Table 3-4 XA_MP3ENC_CONFIG_PARAM_BITRATE subcommand

Subcommand	XA_MP3ENC_CONFIG_PARAM_BITRATE
Description	This command sets the bit rate at which the current encoder operates.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure i_cmd XA_API_CMD_SET_CONFIG_PARAM i_idx XA_MP3ENC_CONFIG_PARAM_BITRATE pv_value &bit_rate – Pointer to PCM word size
Restrictions	The value should be one of the values stated in references ^[1] and ^[2] . Otherwise, it is changed to the next higher bit rate, and the caller is informed of the change through a non-fatal error. The changed value may be obtained after initialization. Note the units are in kHz, for example, 320.

Example

```

unsigned int bitrate;
res = (*api_func)(api_obj,
                  XA_API_CMD_SET_CONFIG_PARAM,
                  XA_MP3ENC_CONFIG_PARAM_BITRATE,
                  (void *) &bitrate);

```

Errors

- Common API Errors
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL

3.3.3 XA_API_CMD_GET_CONFIG_PARAM

Table 3-5 XA_MP3ENC_CONFIG_PARAM_BITRATE subcommand

Subcommand	XA_MP3ENC_CONFIG_PARAM_BITRATE
Description	This command gets the bit rate at which the encoder is operating.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_MP3ENC_CONFIG_PARAM_BITRATE pv_value &bitrate – Pointer to bit rate variable
Restrictions	None

Example

```
unsigned int bitrate;
res = (*api_func)(api_obj,
                  XA_API_CMD_GET_CONFIG_PARAM,
                  XA_MP3ENC_CONFIG_PARAM_BITRATE,
                  (void *) &bitrate);
```

Errors

- Common API Errors
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL

4. Introduction to the Example Test Bench

The supplied sample test bench consists of the following files:

- Test bench source file (found in `test/src`)
 - `xa_mp3_enc_error_handler.c`
 - `xa_mp3_enc_sample_testbench.c`
- Makefile to build the executable (`test/build`)
 - `makefile_testbench_sample`

4.1 Making the Executable Files

This section details how to build the encoder testbench and the required library

Build Encoder Testbench

1. In the command prompt, navigate to the `test/build` directory.
2. Enter the following command:

```
xt-make -f makefile_testbench_sample clean all
```

The `xa_mp3_enc_test` encoder example test bench is built.

Build Encoder Codec Library

Note	If you have source code distribution for the codec package, you must build the MP3 Encoder library before you build the test bench.
-------------	---

1. In the command prompt, navigate to the `build` directory.
2. Enter the following command:

```
$ xt-make clean all install
```

The `xa_mp3_enc.a` library is built and copied to the `lib` directory.

To build and execute the application from .xws based release package, please refer to the *readme.html* file available in the imported application project.

4.2 Running Testbenches

The sample application executable can be run with direct command line options or with a parameter file.

The command line usage is as follows:

```
xt-run xa_mp3_enc_test -infile:<infile> -ofile:<outfile> Cadence is a
pivotal leader in electronic systems design and computational expertise,
using its Intelligent System Design strategy to turn design concepts into
reality. www.cadence.com
      [-br:<bitrate>]
      [-help]
```

where,

- *<infile>* is the name of the input file (wav)
- *<outfile>* is the name of the output file (mp3)
- *<bitrate>* is the bit rate for encoding (default: 128).

Refer to the configuration parameter definitions in the section 3.2 for a full description of their usage.

If command line arguments are not provided, the application reads the commands from the parameter file `paramfilesimple.txt`, which is available in the same directory as the executable.

The syntax of `paramfilesimple.txt` file is as follows:

```
@Start
@Input_path <path to be pre-pended to all input files>
@Output_path <path to be pre-pended to all output files>
<command line 1>
<command line 2>
....
@Stop
```

The MP3 Encoder can be run for multiple test files using different command lines. The syntax for command lines in the parameter file is the same as the syntax for specifying options on the command line. All the `@<command>`s should be at the first column of a line except the `@New_line` command.

Note All the `@<command>`s should be at the first column of a line except the `@New_line` command.

Note All the `@<command>`s are case sensitive. If the command line in the parameter file has to be broken to two parts on two different lines, use the `@New_line` command.

Example:

```
<command line part 1> @New_line
<command line part 2>.
```

Note Blank lines will be ignored.

Note Individual lines can be commented out using `"/"` at the beginning of the line.

5. References

- [1] *ISO/IEC 11172-3 Information technology -- Coding of moving pictures and associated audio for digital storage media at up to about 1,5 Mbit/s -- Part 3: Audio (MPEG-1)*
- [2] *ISO/IEC 13818-3 Information technology -- Generic coding of moving pictures and associated audio information -- Part 3: Audio (MPEG-2)*